

TheKnife

Manuale tecnico

Sommario

Sommario	1
Introduzione	4
Gerarchia delle classi	4
Classi principali	5
theknife.TheKnife	5
menu_log()	5
main_menu()	6
theknife.GestioneFile	7
Struttura dei file	7
parseRiga(String line)	8
Metodi di lettura dei file	8
salvaFileRecensioni(List<List<String>> recensioni)	9
SalvaFilePreferiti(List<List<String>> preferiti)	9
theknife.geoTheKnife	9
public static boolean domicilioEsistente(String domicilio)	9
public static float[] getLatitudineLongitudine(String indirizzo)	9
public static double calcolaDistanza(double lat1, double lon1, double lat2, double lon2)	10
public static ListaRistorante filtraVicinanza(ListaRistorante listaRistoranti, String indirizzo, int raggio)	10
public static ListaRistorante cercaVicinanza(String indirizzo, int raggio)	10
theknife.GestioneUtenti	10
getUsers()	11
register(String username, String psw, String nome, String cognome, String domicilio, String ruolo)	11
login(String username, String psw)	11
checkUser(String username)	11
checkRuolo(String username)	11
theknife.Guest	12

public String getDomicilio()	12
public String getRuolo()	12
public String getUsername()	12
theknife.Utente	12
visualizzaProfilo(Utente user)	13
getPreferiti(String username)	13
aggiungiPreferiti(String username, String nomeRistorante)	13
checkPreferiti(String username, String nomeRistorante)	13
Metodi get	13
theknife.Ristoratore	14
aggiungiRistorante(String username, String name, String address, String location, String price, String cuisine, String phoneNumber, String url, String webSiteUrl, int award, String greenStar, String facilitiesAndServices, String description, boolean delivery, boolean booking)	14
getRistorantiRistoratore(String username)	14
isProprietario(String username, String nomeRistorante) throws IOException	15
theknife.Ristorante	15
Attributi	15
checkRistoranti(String nomeRistorante)	16
scriviRistorante(String nomeRistorante, ...)	16
Metodi get	16
theknife.ListaRistorante	16
getRistoranti()	17
getTipiCucina()	17
filtraTipologia(ListaRistorante listaRistoranti, String tipologia)	17
filtraDelivery(ListaRistorante listaRistoranti)	17
filtraPrezzo(ListaRistorante listaRistoranti, String prezzo)	17
filtraStelle(ListaRistorante listaRistoranti, int stelle)	17
getDatiRistoranti()	18
theknife.Recensione	18
Attributi	18
checkRecensione(String username, String nomeRistorante)	18

checkRisposta(String usernameUtente, String usernameRistoratore, String nomeRistorante).....	18
aggiungiRecensione(String nomeRistorante, Utente user)	18
modificaRecensione(Utente user, String nomeRistorante)	18
eliminaRecensione(Utente user, String nomeRistorante)	19
scriviRisposta(Recensione recensione, String usernameRistoratore, String risposta)	19
rispondiRecensione(String username, Recensione recensione)	19
getRecensioni(String username).....	19
visualizzaRisposta(String usernameUtente, String nomeRistorante)	19
visualizzaRecensioniUtente(Utente user, LinkedList<List<String>> recensioniRistorante)	20
visualizzaRecensioniRistorante(String nomeRistorante, String username, LinkedList<List<String>> recensioniUtente)	20
getMediaVoti(String nomeRistorante).....	20
visualizzaRiepilogo(String nomeRistorante).....	20
Metodi get	20
Altre classi	20
theknife.Utility	20
 stampaRicerca(ListaRistorante ristoranti, String tipoMenu, GestioneUtenti user, String titolo, boolean mostraAddPreferiti).....	20
theknife.Password	21
 encrypt(String clearValue)	21
 decrypt(String encryptedValue)	21
 toString()	21
 Esempio	22
Eccezioni.....	22
 ErroreLogin	22
 UserAlreadyExists.....	22
 RestaurantAlreadyExists.....	22
 RecensioneAlreadyExists	22
Bibliografia e Sitografia	22

Introduzione

TheKnife è una piattaforma che permette di scoprire e selezionare ristoranti in ogni angolo del mondo. Grazie al proprio sistema di ricerca, gli utenti possono filtrare i locali in base alla posizione, alla tipologia di cucina, alla fascia di prezzo e alle opzioni disponibili, come la prenotazione del tavolo o il servizio da asporto.

Gerarchia delle classi

- theknife.TheKnife (Main)
- theknife.GestioneFile
- theknife.GestioneUtenti
 - theknife.Guest
 - theknife.Utente
 - theknife.Ristoratore
- theknife.ListaRistorante
- theknife.Ristorante
- theknife.Password
- theknife.Recensione
- theknife.geoTheKnife
- java.lang.Throwable (implements java.io.Serializable)
 - java.lang.Exception
 - theknife.ErrorLogin
 - theknife.RecensioneAlreadyExists
 - theknife.RestaurantAlreadyExists
 - theknife.UserAlreadyExists
- theknife.Utility

Classi principali

theknife.TheKnife

La classe *TheKnife* rappresenta **classe principale** del progetto. Gestisce i principali menù e le corrispettive funzioni quali: login, registrazione, visualizzazione dei ristoranti, modifica del raggio di ricerca e gestione del profilo utente o ristoratore. Le funzionalità vengono proposte tramite menù testuali.

Campi

public static Scanner sc

Oggetto per l'acquisizione dell'input utente da console.

public static GestioneUtenti user

Oggetto che rappresenta l'utente attualmente loggato, che può essere di tipo Utente, Ristoratore o Guest.

public static int raggio

Raggio geografico (in km) necessario per la visualizzazione dei ristoranti nelle vicinanze.
Default: 30 km.

menu_log()

Metodo iniziale dell'applicazione, consente di:

- Accedere come Guest (utente non registrato).
- Accedere come utente/ristoratore.
- Registrarsi.
- Uscire dall'applicazione.

Comportamento

Il metodo mostra il menù iniziale e attende l'input dell'utente. L'utente può:

- **Accedere come Guest:**

Viene richiesto il domicilio e viene istanziato un oggetto *Guest*.

- **Accedere come Utente/Ristoratore:**

Vengono richieste le credenziali (username e password); in caso di successo, viene istanziato un oggetto di tipo *Utente* o *Ristoratore* tramite il metodo *Utente.login()*.

- **Registrarsi:**

Vengono richiesti i dati necessari alla registrazione (username, password, nome, cognome, domicilio, ruolo). Dopo aver verificato che l'utente non sia già registrato tramite il metodo `Utente.checkUser()`, viene istanziato un oggetto `Utente` o `Ristoratore` e scritto su `users.csv`.

Al termine del processo viene chiamato `main_menu()`.

Eccezioni

- *IOException*, *RestaurantAlreadyExists*, *RecensioneAlreadyExists*, possono essere lanciate da metodi chiamati internamente.
- *ErroreLogin*: viene lanciata quando il login non va a buon fine (credenziali errate).
- *UserAlreadyExists*: gestita nella fase di registrazione, viene lanciata se un utente già esiste (e quindi la registrazione fallisce).

Complessità stimata

- *Login e registrazione*: $O(n)$, dove n è il numero di utenti in `users.csv`.

main_menu()

Comportamento

Il metodo si occupa della visualizzazione del menù principale dell'applicazione. Il menù è dinamico, ovvero varia a seconda del ruolo (Guest, Utente, Ristoratore), in quanto presentano diverse opzioni.

Funzioni comuni:

- Visualizzare la lista dei ristoranti (con filtri).
- Impostare il raggio di ricerca.
- Uscire dall'applicazione.

Funzioni Utente:

- Visualizzare il profilo.

Funzioni Ristoratore:

- Visualizzare la lista dei propri ristoranti.

Complessità stimata

- Ricerca ristoranti per distanza: $O(n)$, dove n è il numero di ristoranti della lista.
- Applicazione dei filtri: $O(k)$, dove k è il numero di filtri selezionati.
- Visualizzazione ristoranti/ristoranti personali: $O(n)$ dove n è il numero di ristoranti in `preferiti.csv` oppure `ristoratori.csv`.

theknife.GestioneFile

La classe GestioneFile si occupa della gestione dei file csv, lettura e scrittura dei file.

Struttura dei file

FILE	Contenuto	Struttura
users.csv	Contiene gli utenti registrati all'applicazione.	Username,Password,Nome,Cognome,Domicilio,Ruolo
ristoratori.csv	Contiene gli username dei ristoratori e i ristoranti a loro associati.	Username_Ristoratore,"Ristorante"
restaurants.csv	Contiene la lista dei ristoranti.	Nome,Indirizzo,Luogo,FasciaPrezzo,TipologiaCucina,Longitudine,Latitudine,Cell,Url,WebsiteUrl,Award,GreenStar,Servizi,Descrizione,Delivery,Booking
preferiti.csv	Contiene la lista dei ristoranti preferiti degli utenti.	Utente,Nome_Ristorante_Preferito
recensioni.csv	Contiene la lista delle recensioni.	Utente_Recensore,"Nome Ristorante",Valutazione,"Recensione"
recensioni_response	Contiene la lista delle risposte alle recensioni.	Utente_Recensore, "Nome Ristorante", Ristoratore_che_Risponde, "Risposta recensione"

Campi statici

`static Path pathUtenti`, percorso al file users.csv

`static Path pathRistoratori`, percorso al file ristoratori.csv.

`static Path pathRistoranti`, percorso al file restaurants.csv.

`static Path pathRecensioni`, percorso al file recensioni.csv.

`static Path pathRisposteRecensioni`, percorso al file recensioni_responses.csv.

`static Path pathPreferiti`, percorso al file preferiti.csv.

N.B: Per ottenere i percorsi completi si usano i metodi `getPathUtenti`, `getPathRistoratori`, `getPathRistoranti`, `getPathRecensioni`, `getPathRisposteRecensioni`, `getPathPreferiti`.

parseRiga(String line)

Metodo per la lettura righe CSV. Utilizza le virgole per delimitare i campi (oltre che le virgolette), consente di estrarre correttamente i dati dai diversi file.

@return Lista contenente i dati.

Esempio

INPUT	OUTPUT (Lista di stringhe)
mattia,ciao1234\$, "via test rossi 15",utente	[mattia, ciao1234, via test rossi 15, utente]

Complessità: $O(n)$, dove n lunghezza della riga.

Metodi di lettura dei file

Questi metodi vengono utilizzati per leggere diversi tipi di file e restituiscono una lista contenente i dati estratti da ciascun file.

Metodo	File letto
<code>getFilePreferiti()</code>	preferiti.csv
<code>getFileRistoratori()</code>	ristoratori.csv
<code>getFileRecensioni()</code>	recensioni.csv
<code>getFileRisposteRecensioni()</code>	recensioni_responses.csv

Eccezioni

- `FileNotFoundException`, se il file non esiste.

Complessità

$O(n)$, dove n è il numero di righe del file.

salvaFileRecensioni(List<List<String>> recensioni)

Il metodo sovrascrive il file *recensioni.csv* con la lista aggiornata, è utilizzato in caso di eliminazione o modifica delle recensioni

SalvaFilePreferiti(List<List<String>> preferiti)

Il metodo sovrascrive il file *preferiti.csv* con la lista aggiornata, è utilizzato quando un utente rimuove un ristorante dai preferiti.

theknife.geoTheKnife

La classe *geoTheKnife* si occupa di fornire le principali funzioni geografiche dell'applicazione:

- Verifica della validità di un indirizzo (domicilio) tramite l'[API Nominatim](#) di OpenStreetMap.
- Ricavare le coordinate geografiche (latitudine, longitudine) dato un indirizzo.
- Dato un indirizzo, filtrare i ristoranti entro un determinato raggio.

public static boolean domicilioEsistente(String domicilio)

Questo metodo si occupa di verificare se un indirizzo esiste o meno, sfruttando **Nominatim** API di OpenStreetMap.

Il metodo costruisce una richiesta HTTP al servizio **Nominatim** di OpenStreetMap, passando l'indirizzo fornito come parametro.

Riceve una risposta in formato JSON:

- Se contiene almeno un risultato, significa che l'indirizzo esiste e il metodo restituisce **true**.
- Se la risposta è vuota, l'indirizzo non è valido e il metodo restituisce **false**.

Limiti

Il controllo effettuato è “**superficiale**”: verifica unicamente che l'indirizzo esista all'interno del database di OpenStreetMap, **non** garantisce che l'indirizzo sia coerente, completo.

public static float[] getLatitudineLongitudine(String indirizzo)

Questo metodo si occupa di ottenere le **coordinate geografiche** (latitudine e longitudine) corrispondenti a un indirizzo testuale. Viene utilizzato per confrontare distanze tra ristoranti e il dato indirizzo.

Il metodo costruisce una richiesta HTTP al servizio Nominatim di OpenStreetMap, passando l'indirizzo fornito come parametro.

Riceve una risposta in formato JSON e si cercano i campi “**lat**” e “**lon**”, se i valori vengono trovati, vengono estratti, convertiti in float e inseriti in un array “a” così strutturato:

- a[0] → Contiene la *latitudine*.
- a[1] → Contiene la *longitudine*.

Eccezioni

IOException, se si verifica un errore durante la richiesta HTTP.

public static double calcolaDistanza(double lat1, double lon1, double lat2, double lon2)

Metodo utilizzato per calcolare la distanza (semplificata) tra due coordinate.

Formula

$$\sqrt{(lat2 - lat1)^2 + (lon2 - lon1)^2} * 111$$

public static ListaRistorante filtraVicinanza(ListaRistorante listaRistoranti, String indirizzo, int raggio)

Dato un indirizzo, filtra una lista di ristoranti restituendo solo quelli che si trovano entro il raggio specificato.

- Ottiene le coordinate dell’indirizzo.
- Itera su ogni ristorante per calcolare la distanza da tali coordinate.
- Aggiunge alla lista in output solo i ristoranti entro il raggio indicato.

Complessità

O(n), dove n è il numero di ristoranti.

public static ListaRistorante cercaVicinanza(String indirizzo, int raggio)

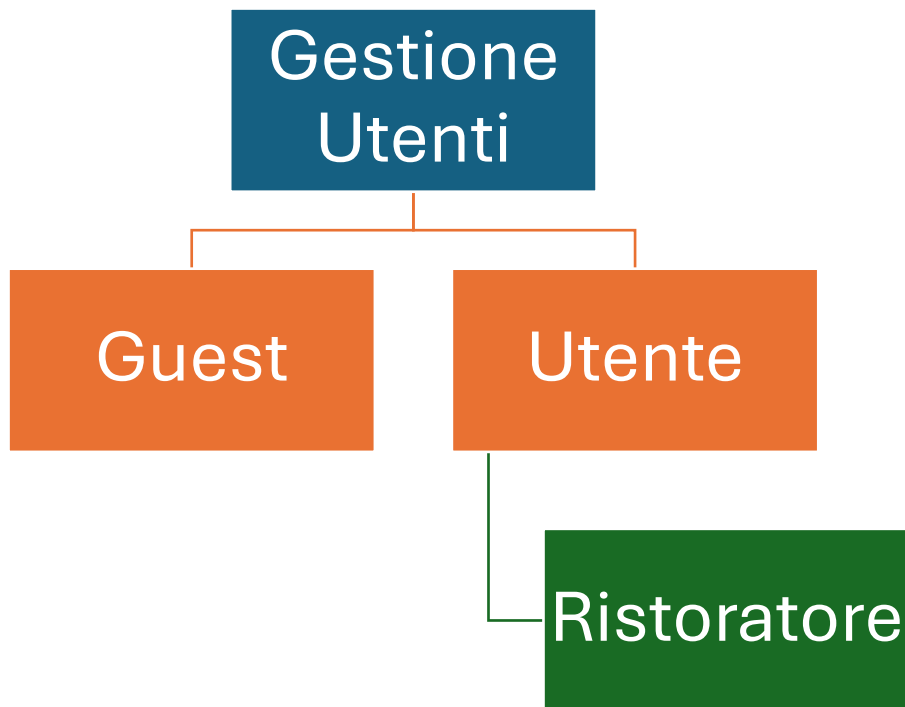
Restituisce la lista di ristoranti entro un certo raggio da un dato indirizzo.

theknife.GestioneUtenti

La classe astratta *GestioneUtenti* è responsabile della gestione delle operazioni comuni sugli utenti. Ne sono degli esempi:

- Lettura utenti da file CSV
- Registrazione nuovi utenti
- Login utenti
- Verifica se un utente è già registrato

La classe presenta metodi per la gestione generale, i metodi specifici sono astratti e implementati nelle sottoclassi *Utente* e *Ristoratore*. Gli utenti sono memorizzati in un file CSV (*users.csv*).



getUsers()

Restituisce una lista di stringhe degli utenti registrati.

Complessità

$O(n)$, n è il numero di righe nel file (il numero di utenti).

register(String username, String psw, String nome, String cognome, String domicilio, String ruolo)

Registra un nuovo utente nell'applicazione e, dopo aver verificato se l'utente esiste *checkUser()* lo scrive nel file *users.csv*.

Complessità

$O(n)$, *checkUser()* legge l'intero file *users.csv*. n è il numero di righe.

login(String username, String psw)

Esegue il login confrontando username e password.

Complessità

$O(n)$, dove n è il numero di utenti in *users.csv*.

checkUser(String username)

Verifica se l'username è presente nel file *users.csv*.

Complessità

$O(n)$, dove n è il numero di utenti in *users.csv*.

checkRuolo(String username)

Verifica e restituisce il ruolo di un utente.

Complessità

$O(n)$, dove n è il numero di utenti in `users.csv`.

theknife.Guest

La classe `Guest` rappresenta un utente non registrato nell'applicazione. Il `Guest` è in grado di utilizzare le funzioni basilari dell'applicazione come la visualizzazione della lista dei ristoranti e impostare il raggio di ricerca. Estende [GestioneUtenti](#).

Attributi

String `username`, fissato a "Guest".

String `domicilio`, richiesto all'utente.

L'oggetto `Guest` viene costruito passando il **domicilio** come parametro al costruttore:

```
Guest g = new Guest("Via Paolo Sarpi 15, Milano");
```

Complessità: $O(1)$.

public String getDomicilio()

Restituisce il domicilio del guest, in tempo $O(1)$.

public String getRuolo()

Restituisce il ruolo del guest, in tempo $O(1)$.

public String getUsername()

Restituisce l'username del guest, in tempo $O(1)$.

theknife.Utente

La classe `Utente` rappresenta un utente registrato nell'applicazione. Estende la classe astratta [GestioneUtenti](#). Implementa i metodi astratti e ulteriori metodi specifici.

Ogni oggetto `Utente` contiene le informazioni personali recuperate dal file `users.csv`, garantendo l'accesso sicuro e strutturato ai dati di un singolo utente. Un oggetto "Utente" viene creato quando l'utente effettua l'accesso oppure si registra correttamente.

Attributi

String `username`, username **UNIVOCO**.

String `nome`, nome dell'utente.

String `cognome`, cognome dell'utente.

String `domicilio`, domicilio dell'utente.

String `ruolo`, corrispondente a "utente".

L'oggetto *Utente* viene costruito passando **username, nome, cognome, domicilio, ruolo ("utente")** come parametri al costruttore:

```
Utente u = new Utente("konoj", "Mattia", "Rotteri", "Via Paolo Rossi, 15, Busto Arsizio, "utente");
```

Complessità: $O(1)$.

visualizzaProfilo(*Utente* user)

Permette all'utente di visualizzare il proprio profilo tramite menù. Nel menù l'utente può visualizzare la sua password, la lista ristoranti preferiti e recensioni da lui scritte.

Complessità: $O(1)$.

getPreferiti(*String* username)

Legge il file *preferiti.csv* e confronta l'username passato come parametro con il primo campo di ogni riga. Se corrispondono, aggiunge il nome del ristorante alla lista. Restituisce infine la lista dei ristoranti preferiti dall'utente.

Complessità: $O(n)$, dove n è il numero di righe di *preferiti.csv*.

aggiungiPreferiti(*String* username, *String* nomeRistorante)

Aggiunge il nome di un ristorante ai preferiti dell'utente scrivendo una nuova riga nel file preferiti.

Format scrittura: nomeutente, nomeRistorante

Complessità: $O(1)$, scrittura nel file.

aggiungiPreferiti(*String* username, *String* nomeRistorante)

Dopo aver controllato se il ristorante è presente nella lista preferiti di un utente (*checkPreferiti()*), lo rimuove e riscrive il file.

Complessità: $O(n)$, *checkPreferiti()* -> $O(n)$, riscrittura del file -> $O(n)$.

checkPreferiti(*String* username, *String* nomeRistorante)

Esamina la lista preferiti di un dato utente e verifica se un dato ristorante è presente tra i preferiti. Restituisce *true* se è presente, *false* altrimenti.

Complessità: $O(n)$, dove n è il numero di righe di *preferiti.csv*.

Metodi get

getNome(), *getCognome()*, *getUsername()*, *getPsw()*, *getDomicilio()*, *getRuolo()*

Restituiscono il valore dell'attributo corrispondente.

theknife.Ristoratore

Rappresenta un utente con ruolo di ristoratore, implementa quindi i metodi per la gestione dei propri ristoranti. Estende [Utente](#).

Attributi

String username, username **UNIVOCO**.

String nome, nome dell'utente.

String cognome, cognome dell'utente.

String domicilio, domicilio dell'utente.

String ruolo, corrispondente a "ristoratore".

L'oggetto *Ristoratore* viene costruito passando **username**, **nome**, **cognome**, **domicilio**, **ruolo** ("ristoratore") come parametri al costruttore:

```
Ristoratore u = new Ristoratore("konoi", "Mattia", "Rotteri", "Via Paolo Rossi, 15, Busto  
Arsizio, "ristoratore");
```

aggiungiRistorante(String username, String name, String address, String location, String price, String cuisine, String phoneNumber, String url, String webSiteUrl, int award, String greenStar, String facilitiesAndServices, String description, boolean delivery, boolean booking)

Aggiunge un nuovo ristorante associato a un ristoratore. Verifica che il ristorante non sia già presente tramite il metodo *checkRistoranti()*. Se l'indirizzo è valido, ne recupera le coordinate geografiche, scrive i dati del ristorante su file tramite *scriviRistorante()*, e infine registra la relazione ristoratore-ristorante nel file dei ristoratori. Se il ristorante è già presente, viene sollevata l'eccezione *RestaurantAlreadyExists*.

Complessità:

Ristorante.checkRistoranti: $O(n)$ con n numero di ristoranti.

geoTheKnife.domicilioEsistente e getLatitudineLongitudine: $O(1)$.

Ristorante.scriviRistorante: $O(1)$.

Scrittura su ristoratori.csv: $O(1)$.

Totale: $O(n)$.

getRistorantiRistoratore(String username)

Restituisce la lista di ristoranti associati al ristoratore *username*.

Legge la lista completa dei ristoranti (*ListaRistorante.getRistoranti*), poi filtra quelli appartenenti al ristoratore leggendo il file *ristoratori.csv*.

Format file ristoratori.csv:

ristoratore, nomeRistorante;

Complessità:

- Scansione lista ristoranti: $O(n)$, n numero totale ristoranti.
- Lettura file ristoranti: $O(m)$, m numero di righe nel file ristoranti.csv.
- **Complessità complessiva $O(m * n)$ nel caso peggiore.**

isProprietario(String username, String nomeRistorante) throws IOException

Verifica se il ristorante *username* è **proprietario** del ristorante *nomeRistorante*.

Restituisce true se è il proprietario, false altrimenti.

theknife.Ristorante

La classe *Ristorante* rappresenta un singolo ristorante. Ad ogni oggetto *Ristorante* sono associati attributi come il nome, l'indirizzo, le coordinate geografiche (latitudine e longitudine), il tipo di cucina etc...

Attributi

String nome;

String indirizzo;

String localita;

String prezzo;

String tipoCucina;

double longitudine;

double latitudine;

String telefono;

String url;

String sitoWeb;

String premio;

String greenStar;

String servizi;

String descrizione;

boolean delivery;

boolean

booking;

L'oggetto *Ristoratore* viene costruito passando **gli attributi** come parametri al costruttore:

```
Ristorante r = new Ristorante(  
    "Trattoria Bellamano", "Via Rossi 10", "Roma", "€€€€", "Italiana",  
    9.1900, 45.4642, "1234567890", "https://instagram.com/bellamano",  
    "https://trattoriabellamano.it", "1 Stella Michelin", "Sì",  
    "Parcheggio gratuito, Animali ammessi", "Locale accogliente",  
    true, false  
);
```

checkRistoranti(String nomeRistorante)

Verifica se un ristorante con il nome specificato (*nomeRistorante*) è già presente all'interno del file *restaurants.csv*. Il metodo legge il file riga per riga e confronta il nome fornito con i nomi dei ristoranti esistenti. Restituisce true se trova il ristorante, in caso contrario, restituisce false.

Complessità: $O(n)$, n è il numero di ristoranti totali.

scriviRistorante(String nomeRistorante, ...)

Aggiunge un nuovo ristorante al file *restaurants.csv* scrivendone i dati in una nuova riga, alla fine del file.

Complessità: $O(1)$, per la scrittura.

visualizzaRistorante(ListaRistorante ristorantiFiltrati, String nomeRistorante)

Stampa le informazioni dettagliate relative a un ristorante specifico, identificato tramite il parametro *nomeRistorante*.

Il ristorante viene cercato all'interno di una lista già filtrata (*ristorantiFiltrati*).

Complessità: $O(n)$, dove n è la lunghezza della ListaRistorante ristorantiFiltrati.

Metodi get

getNome(), *getIndirizzo()*, *getLocalita()*, *getPrezzo()*, *getTipoCucina()*, *getLongitudine()*, *getLatitudine()*, *getTelefono()*, *getUrl()*, *getSitoWeb()*, *getPremio()*, *getGreenStar()*, *getServizi()*, *getDescrizione()*, *isDelivery()*, *isBooking()*

Restituiscono il valore dell'attributo corrispondente.

theknife.ListaRistorante

La classe *ListaRistorante* gestisce una collezione di oggetti di tipo **Ristorante**.

Tra le sue principali funzionalità, permette di caricare la lista di ristoranti da *restaurants.csv*, istanziando un oggetto **Ristorante** per ciascun ristorante.

Inoltre, offre metodi per applicare **filtri** sui ristoranti, come il tipo di cucina, la disponibilità del servizio delivery, la possibilità di prenotazione, la fascia di prezzo, il numero di stelle di valutazione e la località.

getRistoranti()

Restituisce la lista di tutti i ristoranti letti dal file CSV *restaurant.csv*.

Complessità: $O(n)$, con n numero di righe (ristoranti) in *restaurants.csv*.

getTipiCucina()

Restituisce la lista **UNICA** di tutti i tipi di cucina letti dal file CSV *restaurant.csv*.

Complessità: $O(n)$, con n numero di righe (ristoranti) in *restaurants.csv*.

filtraTipologia(ListaRistorante listaRistoranti, String tipologia)

Restituisce un nuovo oggetto *ListaRistorante*, ovvero la lista di ristoranti che presentano il tipo di cucina richiesto.

Complessità: $O(n)$, con n numero di ristoranti nella lista.

filtraDelivery(ListaRistorante listaRistoranti)

Restituisce un nuovo oggetto *ListaRistorante*, ovvero la lista di ristoranti che offrono il servizio di delivery.

Complessità: $O(n)$, con n numero di ristoranti nella lista.

filtraBooking(ListaRistorante listaRistoranti)

Restituisce un nuovo oggetto *ListaRistorante*, ovvero la lista di ristoranti che offrono il servizio di prenotazione.

Complessità: $O(n)$, con n numero di ristoranti nella lista.

filtraPrezzo(ListaRistorante listaRistoranti, String prezzo)

Restituisce un nuovo oggetto *ListaRistorante*, ovvero la lista di ristoranti che appartengono alla data fascia di prezzo ("€", "€€", ..).

Confronta la lunghezza della stringa prezzo (es. "€€") con quella del ristorante.

Complessità: $O(n)$, con n numero di ristoranti nella lista.

filtraStelle(ListaRistorante listaRistoranti, int stelle)

Restituisce un nuovo oggetto *ListaRistorante*, ovvero la lista di ristoranti presentano la media di stelle richiesta.

Usa *Recensione.getMediaVoti(nomeRistorante)* per ottenere la media voti.

Complessità: $O(n)$, con n numero di ristoranti.

getDatiRistoranti()

Restituisce la lista di ristoranti.

Complessità: $O(1)$.

theknife.Recensione

La classe *Recensione* gestisce le recensioni scritte dagli utenti.

Attributi

String nomeRistorante Nome del ristorante recensito.

String usernameUtente (String): Username dell'utente che ha scritto la recensione.

int valutazione, numero di stelle della recensione (da 1 a 5).

String contenuto, Testo scritto dell'opinione o commento.

L'oggetto *Recensione* viene costruito passando **gli attributi** come parametri al costruttore:

```
Recensione r = new Recensione("konoï", "Pizzeria da Mario", 5,"Bellissimo  
Ristorante!");
```

checkRecensione(String username, String nomeRistorante)

Verifica se, dato l'username di un utente, ha già lasciato una recensione per il ristorante nomeRistorante. Restituisce true se la recensione esiste, false altrimenti.

Complessità: $O(n)$, dove n è il numero di recensioni nel file recensioni.csv.

checkRisposta(String usernameUtente, String usernameRistoratore, String nomeRistorante)

Verifica se un ristoratore (*usernameRistoratore*) ha già risposto alla recensione di *usernameUtente*, per il ristorante *nomeRistorante*.

Complessità: $O(n)$, dove n è il numero di righe nel file recensioni_responses.csv.

aggiungiRecensione(String nomeRistorante, Utente user)

Permette ad un utente di scrivere una nuova recensione per un ristorante. Se l'utente ha già scritto una recensione per quel dato ristorante (*nomeRistorante*), lancia l'eccezione *RecensioneAlreadyExists*.

Complessità: $O(n)$, *checkRecensioni()*.

modificaRecensione(Utente user, String nomeRistorante)

Permette di modificare una recensione esistente eliminandola e richiedendo la nuova recensione all'utente.

Complessità: $O(n)$, `checkRecensioni()` e `eliminaRecensione()`.

eliminaRecensione(Utente user, String nomeRistorante)

Elimina la recensione scritta da un utente per un dato ristorante (*nomeRistorante*).

Complessità: $O(n)$, dove n è il numero di recensioni nel file `recensioni.csv`.

scriviRisposta(Recensione recensione, String usernameRistoratore, String risposta)

Registra una nuova risposta a una recensione scrivendola nel file `recensioni_responses.csv`.

Complessità: $O(1)$, scrittura sul file.

rispondiRecensione(String username, Recensione recensione)

Consente a un ristorante di rispondere a una recensione, se non ha già risposto.

Nota bene: il ristorante può scrivere massimo una risposta per recensione!

Complessità: $O(n)$, `checkRecensioni()`.

getRecensioni(String username)

Restituisce una lista con tutte le recensioni scritte da un utente specifico, identificato dall'`username`.

- Legge il file `recensioni.csv`.
- Confronta il campo `username` con l'`username` passato come parametro.
- Ritorna la lista di recensioni trovate.

Complessità: $O(n)$, dove n è il numero di recensioni in `recensioni.csv`.

getRecensioniRistorante(String nomeRistorante)

Restituisce tutte le recensioni relative a un dato ristorante (*nomeRistorante*).

- Legge il file `recensioni.csv`.
- Confronta il campo `nomeRistorante` con il `nomeRistorante` passato come parametro.
- Ritorna la lista di recensioni trovate.

Complessità: $O(n)$, dove n è il numero di recensioni in `recensioni.csv`.

visualizzaRisposta(String usernameUtente, String nomeRistorante)

Legge le risposte alle recensioni, se trova una risposta all'utente `usernameUtente`, per il ristorante `nomeRistorante`, restituisce la risposta del ristorante alla recensione, nel corrispettivo menù testuale.

Complessità: $O(n)$, dove n è il numero di risposte alle recensioni in `recensioni_responses.csv`.

visualizzaRecensioniUtente(Utente user, LinkedList<List<String>> recensioniRistorante)

Consente a un utente di visualizzare le proprie recensioni, modificarle ed eliminarle.

Complessità: $O(n)$.

visualizzaRecensioniRistorante(String nomeRistorante, String username, LinkedList<List<String>> recensioniUtente)

Permette di visualizzare tutte le recensioni relative a un ristorante specifico e, se l'utente è un ristoratore proprietario di quel ristorante, abilita la possibilità di rispondere alle recensioni stesse.

Complessità: $O(n)$.

getMediaVoti(String nomeRistorante)

Calcola la media dei voti dati a un ristorante dalle recensioni degli utenti.

Complessità: $O(n)$, dove n è il numero di recensioni in recensioni.csv.

visualizzaRiepilogo(String nomeRistorante)

Stampa un riepilogo delle recensioni di un ristorante, includendo il numero totale di recensioni e la media dei voti.

Complessità: $O(n)$, getRecensioniRistorante() e getMediaVoti().

Metodi get

getDesc, getNomeRistorante, getUser, getValutazione

Restituiscono il valore dell'attributo corrispondente.

Altre classi

Classi utili per il funzionamento dell'applicazione.

theknife.Utility

Fornisce metodi di utilità generali per l'applicazione, tra cui il metodo dinamico di stampa, la gestione dei menù (stampaRicerca, cercaFiltri, menuAggiungiRistorante etc.), la stampa del logo e la pulizia del terminale.

stampaRicerca(ListaRistorante ristoranti, String tipoMenu, GestioneUtenti user, String titolo, boolean mostraAddPreferiti)

Questo metodo è fondamentale per il corretto funzionamento dell'applicazione. Viene utilizzato per mostrare all'utente una lista di ristoranti con paginazione, permettendo di navigare tra le pagine, visualizzare le informazioni di un ristorante specifico, aggiungere un nuovo ristorante (se l'utente è un ristoratore), gestire i preferiti, leggere e scrivere recensioni, e ottenere un riepilogo di un ristorante (se l'utente ne è proprietario).

Funzionamento:

- In base al tipo di menù passato come stringa (*filtri*, *preferiti* o *ristoratore*), seleziona la lista di ristoranti da visualizzare.
- Mostra a schermo massimo 10 ristoranti per pagina, con comandi per andare avanti (>), indietro (<) o uscire.
- Consente all'utente di visualizzare le informazioni dettagliate di un ristorante tramite il comando **CERCA** <nomeRistorante>.
- Se l'utente è un ristorante, permette di aggiungere nuovi ristoranti tramite il comando **AGGIUNGI**.
- Quando si seleziona un ristorante, è possibile:
 - Visualizzare tutte le recensioni.
 - Aggiungere o rimuovere il ristorante dai preferiti (se l'utente non è guest e la funzione è abilitata ovvero `mostraPreferiti = true`).
 - Scrivere una recensione (se l'utente non è guest e non ha già recensito il ristorante).
 - Visualizzare un riepilogo delle recensioni (se l'utente è il *proprietario* del ristorante).

Complessità: $O(n)$.

theknife.Password

La classe *Password* rappresenta una password cifrata utilizzando una semplice cifratura basata sulla lunghezza della stringa. Ogni carattere della password originale viene traslato di un valore pari alla lunghezza della stringa stessa.

Password p = new Password("ciao1234\$")

encrypt(String clearValue)

Cifra una stringa traslando ogni carattere di un numero di posizioni pari alla lunghezza della stringa.

Complessità: $O(n)$, dove n è la lunghezza della stringa.

decrypt(String encryptedValue)

Decifra una stringa cifrata traslando indietro ogni carattere di un numero di posizioni pari alla lunghezza della stringa cifrata.

Complessità: $O(n)$, dove n è la lunghezza della stringa.

toString()

Restituisce la password cifrata come *String*.

Complessità: $O(1)$.

Esempio

CLEAR VALUE	ENCRYPTED
Ciao1234\$	kqiw9;:<

Ogni lettera viene traslata di 9 posizioni.

Eccezioni

ErroreLogin

Eccezione che viene lanciata per segnalare credenziali errate inserite durante il login da parte dell'utente.

UserAlreadyExists

Eccezione che viene lanciata quando si tenta di registrare un utente con un username già esistente.

RestaurantAlreadyExists

Eccezione che viene lanciata quando si cerca di aggiungere un ristorante già esistente.

RecensioneAlreadyExists

Eccezione che viene lanciata quando si cerca di aggiungere una recensione già esistente.

Bibliografia e Sitografia

Dai fondamenti agli oggetti, Giovanni Pighizzini, Mauro Ferrari.

[API Nominatim](#), di OpenStreetMap.